MAE 3780 Final Report

Team 7: Ayanna Fulga (asf225) / Aidan Friedman (ajf246) / Samantha Pochet (sep244)

1. Robot Design and Strategy Overview

For the MAE 3780 "Cube Craze" Robot Competition, we built "Bus", a robot designed to autonomously collect and retain as many colorful cubes as possible in one minute. The robot's overall mechanical design is quite simple, utilizing the chassis and wheels provided from the lab. On the exterior, the robot's outer frame is built entirely out of cardboard, which is used to create the robot's 8x8'' perimeter. Cardboard is additionally used to line the exterior of the chassis and wheels to direct the cubes away from the main body of the robot and into specific collection slots to the side of the robot. Preventing the cubes from contacting the wheels or chassis was essential both to avoid damaging components and to ensure that interactions with the cubes did not affect the robot's overall motion. Finally, pieces of cardboard were also used to connect the exterior cardboard perimeter to the cardboard lining the chassis to reduce flexure and ensure that the robot maintained a constant 8x8'' profile throughout the entire duration of the competition in compliance with competition rules.

In terms of sensors and actuators, our robot implemented two main servos connected to the wheels, one color sensor mounted in the middle of the robot, and two QTI sensors, one mounted on each side of the robot (left and right with respect to the wheels). Both the color sensors and the QTI sensors were mounted as close to the ground as possible while still maintaining sufficient driving clearance in order to minimize sensor noise and maximize the accuracy and consistency of the readings sent to the Arduino. While the color sensor was not implemented in our control logic after Milestone 3, it remained installed and wired up for the competition as we found the white LED illumination coming from the color sensor helped improve the accuracy of the nearby QTI sensor readings. A 4-cell AA battery pack was used to power the drive servos, and a 9V battery was used to directly power the Arduino board. All wiring and electronic integration was done through jumper wires and breadboard circuitry.

On theme with the mechanical and electrical design, our robot's software design was also relatively simple. Our primary goal was to use the QTI sensors to ensure that the robot remained on the board at all times while employing pseudo-random motion to "sweep" the board, in theory maximizing both area coverage and the number of cubes collected within one minute. At startup, the robot first drives forward for a few seconds, until it is in the center of the board. After this initial movement, the robot performs a pseudo-random 90-degree turn either to the left or right. Since most of the cubes are located around the center of the board, this helps maximize the amount of cubes collected. The program then enters an infinite control loop

in which the QTI sensors are continuously sampled. If a sensor reading exceeds a threshold value, the software interprets this as detection of the board edge. During normal operation, the robot drives forward in short pulses at a relatively slow speed. This slower movement was intentionally chosen to give the QTI sensors more opportunities to detect the edges of the board and to reduce the likelihood of cubes being knocked out of the robot's perimeter during the match.

To keep the robot on the board, the software implemented edge-detection logic, where if the left QTI sensor detects the black border, the robot briefly turns right to return toward the interior of the board, and if the right sensor detects the border, it turns left. Additionally, if both QTI sensors detect the border, the robot initiates a short reverse followed by a large turn utilizing one wheel in order to ensure that the collected cubes remain in the perimeter of the robot. To improve coverage of the board, the magnitude of these turns was varied using a simple pseudo-random counter, reducing repetitive motion patterns. Logic was also implemented so that the robot turned the correct way when it reached a corner: for example, if the left QTI sensor had been detecting black leading up to the full edge detection, the robot would turn right, and vice versa.

Overall, our robot's mechanical, electrical, and software design prioritized simplicity in favor of reliable and well-characterized behavior as well as effective cube-collection on the board. With our robust control logic demonstrated during lab testing, we were hopeful that our design would hold up well even with a competitor on the board.

2. Design Process Reflection

As we began working on completing the milestones, we realized that our group's initial design plans were a tad more complicated than necessary. This new goal of a simple and effective overall design resulted in our group prioritizing completion of the milestones as a primary task, and making our final competition robot off of adjustments to our milestone 4 base design. This strategy worked in our favor, as this simplified design allowed us to reliably collect blocks without having to worry about code errors or extraneous mechanical issues on competition day.

3. Competition Analysis

Our robot performed well overall during the tournament, winning two matches largely because of its reliable edge-avoidance logic, which consistently kept it within the playable area while continuing to search for cubes. Most of our losses were narrow and resulted from unexpected behavior by other robots, such as where the other robot would push cubes off the course where our robot could not retrieve them. We also encountered an unexpected issue with battery charge level: the robot behaved differently with fully charged batteries than it had during testing, causing faster movement and a resulting search pattern that differed from what we originally programmed. The robot's main strengths were its reliable edge detection and logic, while its main weaknesses were limited adaptability to changing match conditions and sensitivity to motor speed variations.

4. Conclusions

While our robot only won two out of its six matches, we were pleased with our performance and with the project as a whole. Overall, our robot's edge-detection and gameplay logic were robust, and coupled with a simple yet sound mechanical design made for quite a reliable robot. Looking back, however, we definitely erred on the side of caution when it came to our design, and were we to do this project over again with the same budget and constraints, we would definitely take some bigger risks in terms of design and control logic. First, we would incorporate a deployable mechanism to increase our block-collecting area rather than staying in one configuration throughout the whole match. There were a number of teams that implemented mechanisms to enhance block-collecting, and when done right, we saw that they can be a huge advantage to have on the board. Additionally, we would likely increase the speed of our robot. If we took the time to implement more robust edge-detection logic in our software, we would be able to run the robot much faster without compromising reliability. To students doing this project next year, our advice would be to take just one risk with your initial design, whether that be mechanical, electrical, or software related. Make sure, however, that you have a backup solution if that doesn't end up working out.

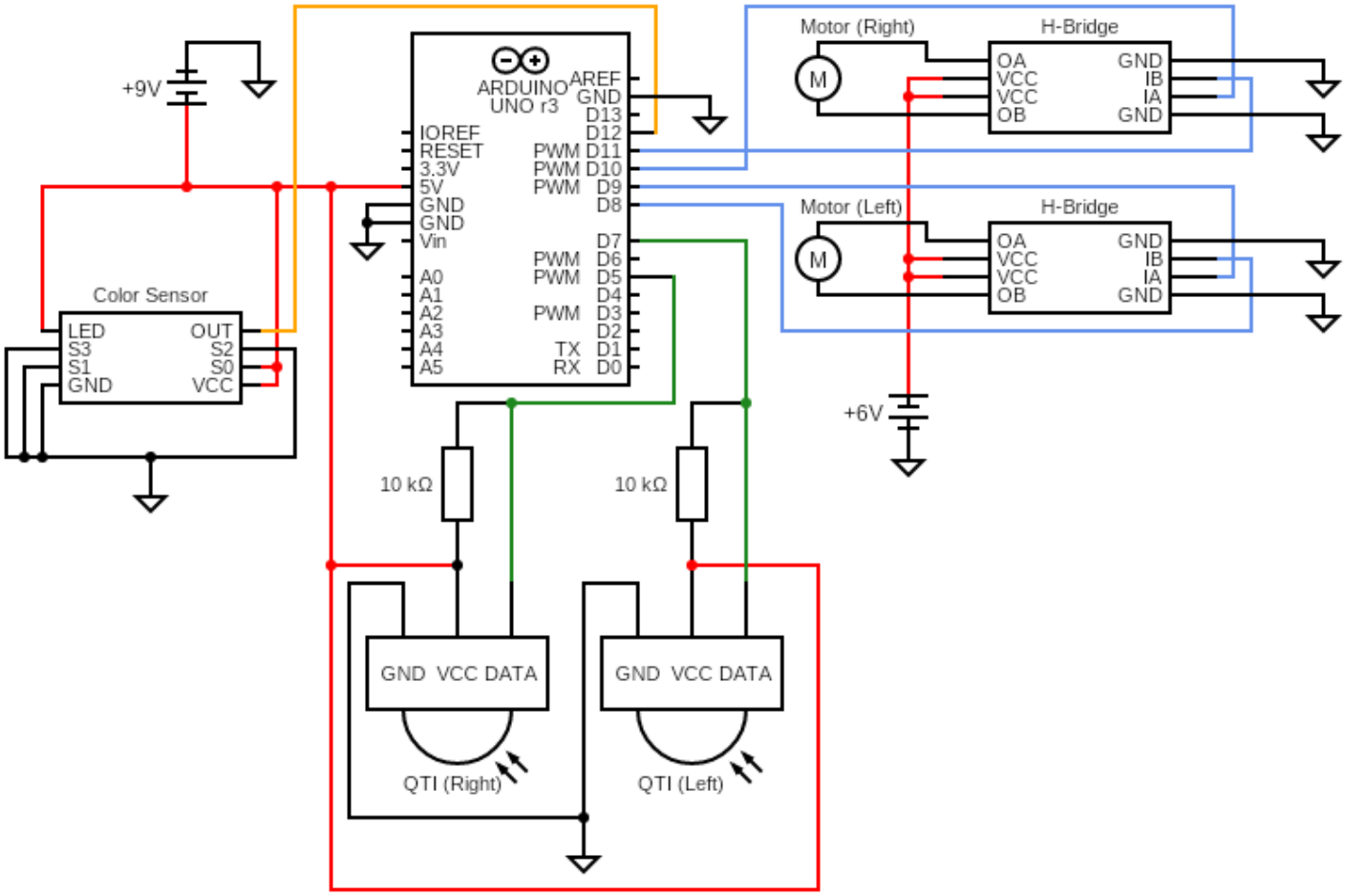
Appendix A

Bill of Materials

Item	Quantity	Price per Quantity	Total Price
Cardboard (McMaster - 20585T22)	5	$11.14	$11.14
QTI	2	$2	$4
L9110H H-bridge	1	$1	$1

Appendix B

Circuit Diagram



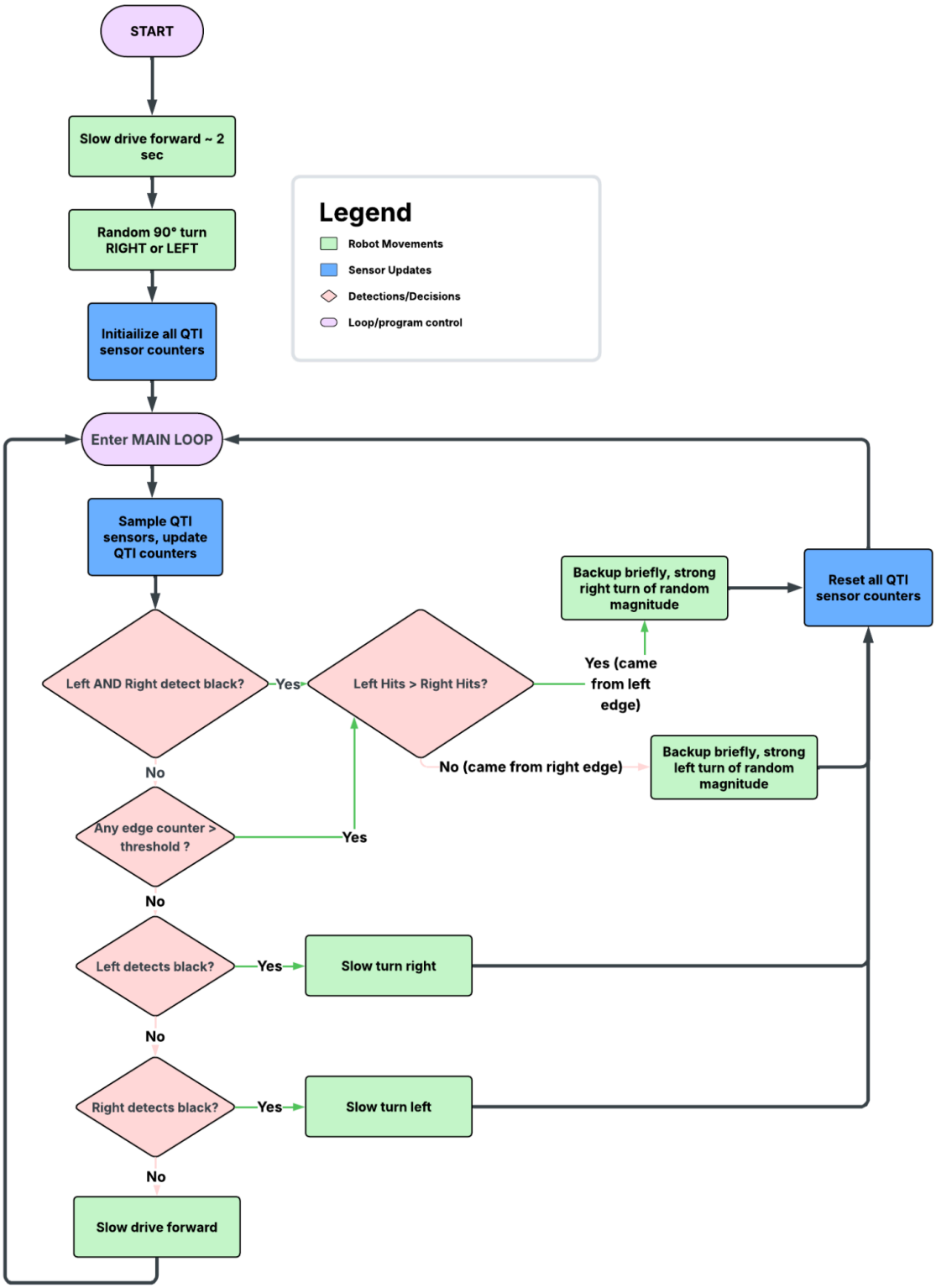
Appendix C

CAD Files & Drawings

N/A

Appendix D

Flowchart



Appendix E

Arduino Code

```
#include <avr/io.h>      // AVR I/O registers
#include <util/delay.h>  // busy-wait delays

// ---------------- STOP ----------------
void stop() {
   PORTB &= ~((1 << PB0)|(1 << PB1)|(1 << PB2)|(1 << PB3));  // all motor pins LOW
}

// ---------------- BACKUP ----------------
void backup_brief() {
   PORTB &= ~(1 << PB0);  // left A LOW
   PORTB |= (1 << PB1);   // left B HIGH = reverse
   PORTB &= ~(1 << PB2);  // right A LOW
   PORTB |= (1 << PB3);   // right B HIGH = reverse
   _delay_ms(150);        // reverse 150 ms
   stop();
   _delay_ms(20);         // settle
}

// ---------------- MOTOR CONTROL ----------------
void forward_slow() {
   PORTB |= (1 << PB0);   // left A HIGH
   PORTB &= ~(1 << PB1);  // left B LOW = forward
   PORTB |= (1 << PB2);   // right A HIGH
   PORTB &= ~(1 << PB3);  // right B LOW = forward
   _delay_ms(20);         // drive 20 ms
   stop();
   _delay_ms(3);          // dead-time = slow speed
}

void turn_left_slow() {
   PORTB &= ~(1 << PB0);  // left off
   PORTB &= ~(1 << PB1);  // left off
   PORTB |= (1 << PB2);   // right A HIGH
   PORTB &= ~(1 << PB3);  // right B LOW = forward
```

```
    _delay_ms(12);         // right only = pivot left
    stop();
    _delay_ms(12);         // 50% duty
}

void turn_right_slow() {
    PORTB |= (1 << PB0);   // left A HIGH
    PORTB &= ~(1 << PB1);  // left B LOW = forward
    PORTB &= ~(1 << PB2);  // right off
    PORTB &= ~(1 << PB3);  // right off
    _delay_ms(12);         // left only = pivot right
    stop();
    _delay_ms(12);         // 50% duty
}

// ---------------- STRONG ESCAPE TURN ----------------
void strong_turn_left(int reps) {
    backup_brief();                    // clear edge
    for (int i = 0; i < reps; i++) {
        turn_left_slow();              // reps x 24 ms = turn arc
    }
}

void strong_turn_right(int reps) {
    backup_brief();                    // clear edge
    for (int i = 0; i < reps; i++) {
        turn_right_slow();             // mirror of left side
    }
}

// ---------------- QTI ----------------
int read_qti(uint8_t pin) {

    DDRD |= (1 << pin);    // pin OUTPUT
    PORTD |= (1 << pin);   // HIGH = charge RC
    _delay_us(10);         // fully charge

    DDRD &= ~(1 << pin);   // pin INPUT = RC discharges
```

```
    int count = 0;

    while ((PIND & (1 << pin)) && count < 1000) {
        count++;            // count 10 us ticks until pin falls
        _delay_us(10);      // low = black (fast), ~999 = white (slow)
    }

    return count;           // threshold checked
}

// ---------------- MAIN ----------------
int main(void) {

    DDRB |= (1 << PB0)|(1 << PB1)|(1 << PB2)|(1 << PB3);  // motor pins as OUTPUT

    // -------- STARTUP: drive forward ~2 seconds --------
    for (int i = 0; i < 87; i++) {   // 87 x 23 ms = ~2 s
        forward_slow();
    }

    // -------- STARTUP: random 90-degree turn --------
    // use a simple counter seeded by loop count above for random
    uint8_t pseudo_rand = 90;         // seed (even, so always turns right)
    if (pseudo_rand % 2 == 0) {       // always true at startup
        for (int i = 0; i < 38; i++) turn_right_slow();   // 38 pulses = ~90 degrees
    } else {
        for (int i = 0; i < 38; i++) turn_left_slow();    // unreachable (seed=90)
    }

    int edge_counter = 0;   // consecutive loops on edge
    int left_hits = 0;      // cumulative left edge hits
    int right_hits = 0;     // cumulative right edge hits
    int turn_count = 0;     // total escapes (increases arc)

    while (1) {

        pseudo_rand++;  // pseudo-random each loop
```

```
int left_black  = (read_qti(PD7) >= 999);  // left sensor on edge
int right_black = (read_qti(PD5) >= 999);  // right sensor on edge

// -------- TRACK EDGE CONTACT --------
if (left_black)  left_hits++;
if (right_black) right_hits++;

if (left_black || right_black) {
   edge_counter++;   // on edge, extend dwell
} else {
   edge_counter = 0; // clear, reset dwell
}

// larger arc after 4+ escapes, randomized
int reps = (turn_count >= 4) ? 70 + (pseudo_rand % 31) : 55 + (pseudo_rand % 11 + 15);

// -------- CORNER CASE --------
if (left_black && right_black) {  // both on edge = corner

   if (left_hits > right_hits) {
      // came from left edge -> go right
      strong_turn_right(reps);
   } else {
      // came from right edge -> go left
      strong_turn_left(reps);
   }

   turn_count++;
   left_hits = 0;
   right_hits = 0;
   edge_counter = 0;

   continue;
}

// -------- ESCAPE EDGE LOCK --------
if (edge_counter > 40) {  // stuck on edge, force escape
```

```
        edge_counter = 0;

        if (left_hits > right_hits) {
            strong_turn_right(reps);  // left dominant -> escape right
        } else {
            strong_turn_left(reps);   // right dominant -> escape left
        }

        turn_count++;
        left_hits = 0;
        right_hits = 0;

        continue;
    }

    // -------- NORMAL EDGE REACTION --------
    if (left_black) {
        turn_right_slow();  // left on edge -> steer right
        continue;
    }
    else if (right_black) {
        turn_left_slow();   // right on edge -> steer left
        continue;
    }

    // -------- DEFAULT MOVEMENT --------
    forward_slow();  // no edge, advance
  }
}
```